

Angular Learning Series

Topics 20, 28, 29, 30, 31, 32

Standalone Components | Template Syntax | Control Flow |

Built-in Directives | Content Projection | Signal Queries

Modern Angular v17+ Syntax Throughout

TOPIC 20 — Standalone Components

Category: Angular Core | Difficulty: Beginner-Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

- Difficulty: Beginner-Intermediate — the concept is simple; the mental shift from NgModules is the only hurdle
- Angular Relevance: Critical — standalone is the default Angular architecture from v17+; everything else builds on top of this

Section 1 — Explanation

The Lego Box Analogy

Old Angular worked like a Lego set where every piece had to be registered in a central box (NgModule) before you could use it. You wanted to use a piece in room A? Register it in the box. Room B too? Same box, or a new box, or import the box that has that piece.

Standalone components throw the box away. Each Lego piece now carries its own list of what it needs. Want to use a pipe inside a component? Just import it directly in that component. Done. No shared box. No registration ceremony.

Technically: a standalone component is a self-describing unit. It declares its own imports, meaning it does not need to belong to any NgModule to function. Angular resolves its dependencies locally.

Section 2 — Modern Angular Pattern (v17+)

```
@Component({
  selector: 'app-root',
  standalone: true,           // default in Angular v17+
  imports: [...],            // declare what THIS component needs
  template: `...`
})
```

From Angular v17 onwards, **standalone: true is the default**. The Angular CLI generates standalone components automatically. You no longer write `standalone: true` explicitly — it is assumed unless you opt out.

Migration path from NgModules

- Remove declarations array from NgModule
- Add `standalone: true` to each component
- Move NgModule imports into each component's own imports array
- Delete NgModule when all components are migrated

Section 3 — Code Example

```
// Standalone component – complete and runnable
```

```

import { Component, signal } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-counter',
  standalone: true,
  imports: [CommonModule],    // this component imports what IT needs
  template: `
    <h2>Count: {{ count() }}</h2>
    <button (click)="increment()">+</button>
  `
})
export class CounterComponent {
  count = signal(0);
  increment() { this.count.update(v => v + 1); }
}

// Bootstrapping a standalone app – main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent);
// No AppModule. No platformBrowserDynamic().bootstrapModule().

```

Section 4 — Before & After Refactor

```

// BEFORE – NgModule pattern
@NgModule({
  declarations: [AppComponent, CounterComponent],
  imports: [BrowserModule, CommonModule],
  bootstrap: [AppComponent]
})
export class AppModule {}

// AFTER – Standalone pattern
@Component({
  standalone: true,
  imports: [CommonModule],
  ...
})
export class CounterComponent {}

// main.ts
bootstrapApplication(AppComponent);

```

Why it matters: Eliminates the module layer entirely. Dependency scope is co-located with the component that needs it — far easier to trace, test, and tree-shake.

Section 5 — Common Mistakes

Mistake 1 — Importing a component but forgetting standalone: true

Only standalone components can be directly imported by other components. Non-standalone ones still need a module. Angular throws a build error — it cannot resolve the import.

Mistake 2 — Importing at the wrong level

Putting a pipe/directive import in AppComponent and expecting child components to get it "for free." Standalone has no shared inheritance — each component declares its own imports. Runtime template error: the pipe will not be found in child components.

Mistake 3 — Still bootstrapping with AppModule

Running bootstrapModule(AppModule) even after converting to standalone. Results in a hybrid app with subtle DI issues. Fix: use bootstrapApplication(AppComponent) in main.ts.

Section 6 — Do's and Don'ts

- DO: Use standalone: true on every new component
- DON'T: Mix NgModule and standalone without a clear migration plan
- DO: Import only what each component needs
- DON'T: Import CommonModule everywhere "just in case"
- DO: Use bootstrapApplication() for new apps
- DON'T: Use bootstrapModule(AppModule) in a standalone project
- DO: Import RouterModule or individual pipes directly in the component
- DON'T: Rely on a parent component's imports to "trickle down"

Section 7 — Interview Prep

Question 1

Question: "What problem do standalone components solve, and how does that change how you structure an Angular application?"

Strong answer: NgModules were a declaration boundary with no real runtime role — they existed to tell the Angular compiler what exists. Standalone components remove that indirection. Each component is now a self-contained compilation unit. It imports exactly what it uses. This makes tree-shaking more effective, testing easier (no TestBed module setup), and mental overhead lower. App structure shifts from module-centric to component-centric.

Weak answer: "It means you don't need app.module.ts anymore."

Level: Mid to Senior

Question 2

Question: "How do you provide a service that should be singleton across a standalone Angular app with no root module?"

Strong answer: Two ways. providedIn: 'root' on the service still works — Angular registers it at the application root regardless of whether a module exists. For app-wide providers that are not services (interceptors, guards), pass them into bootstrapApplication() via the providers array: bootstrapApplication(AppComponent, { providers: [provideHttpClient()] }).

Weak answer: "You inject it in the component."

Level: Senior

Section 8 — Quick Reference Card

3-line definition

A standalone component owns its own import list. No NgModule required to declare or use it. Self-contained compilation unit.

Syntax at a glance

```
@Component({ standalone: true, imports: [CommonModule, RouterLink], ... })
```

- When to use: Always — for any new Angular component from v17+
- When not to use: When maintaining an existing NgModule-based codebase where migrating is not worth the risk mid-feature

The one rule that matters most

Every standalone component imports what it needs itself — nothing trickles down from a parent.

TOPIC 28 — Angular Template Syntax

Category: Angular Core | Difficulty: Beginner-Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

- Difficulty: Beginner-Intermediate — the individual syntaxes are simple, but knowing which binding to use and why requires understanding Angular's data flow model
- Angular Relevance: Critical — template syntax is the bridge between TypeScript component logic and the HTML the user sees; every Angular component uses these four mechanisms

Section 1 — Explanation

The Dashboard Analogy

A car dashboard shows engine data — you can read it but cannot change the engine speed by touching the gauge. That's interpolation and property binding — data flows one way, from the engine (component class) to the display (template).

The accelerator pedal does the opposite — you push it and it sends a signal back to the engine. That's event binding — user action in the template sends data back to the component class.

The cruise control dial does both — you set a speed, the car holds it, and the dial always reflects the current speed. That's two-way binding — data flows in both directions simultaneously.

The Four Mechanisms

Mechanism	Syntax	Direction	Use Case
Interpolation	{{ expression }}	Class -> Template	Display text values
Property Binding	[property]="expression"	Class -> Template	Set DOM/component props
Event Binding	(event)="handler()"	Template -> Class	Respond to user actions
Two-Way Binding	[(ngModel)]="property"	Both	Form inputs

Key insight: Square brackets [] mean "reading from the class." Parentheses () mean "sending to the class." Banana-in-a-box [()] is both combined.

Interpolation {{ }}

Evaluates a TypeScript expression, converts to string, inserts into DOM as text. Only produces text — cannot set DOM attributes that expect non-string values like [disabled], [src], [class].

Property Binding [property]="expression"

Sets a DOM element property or child component's input() property to a TypeScript expression's value — evaluated as JavaScript, not a string. Important: HTML attributes initialise the DOM; properties are the live JavaScript values. [attr.colspan]="n" is attribute binding; [value]="n" is property binding.

Event Binding (event)="handler(\$event)"

Listens for a DOM event or custom output event and calls a class method. \$event carries the event payload — either a native DOM Event object or whatever type the output() emitter was defined with.

Two-Way Binding [(ngModel)]="property"

Combines a property binding and an event binding. Under the hood, `[(ngModel)]` is exactly: `[(ngModel)]="username" (ngModelChange)="username = $event"`. Works with any component that exposes a matching `input()` named `X` and an `output()` named `xChange` — the banana-in-a-box convention.

Section 2 — Modern Angular Pattern (v17+)

All four binding mechanisms are unchanged in v17+. What changes is what you bind to:

- `input()` signal functions replace `@Input()` decorators — property binding syntax `[prop]="val"` is identical, but reading the value in the class uses `this.prop()` instead of `this.prop`
- `output()` functions replace `@Output()` + `EventEmitter` — event binding syntax `(event)="handler($event)"` is identical
- `@if` / `@for` / `@switch` replace `*ngIf` / `*ngFor` / `*ngSwitch` — no `CommonModule` import needed
- For standalone components, import `FormsModule` directly in the component's imports array for `ngModel`

Banana-in-a-box with modern syntax

```
// Child component
export class CounterComponent {
  count      = input(0);
  countChange = output<number>();

  increment(): void { this.countChange.emit(this.count() + 1); }
  decrement(): void { this.countChange.emit(this.count() - 1); }
}

// Parent usage – syntax unchanged
// <app-counter [(count)]="myCount"></app-counter>
```

Section 3 — Code Example

```
// Plain TS – what Angular automates
class UserCard {
  name = 'Alice';
  isActive = true;
  greet() { console.log(`Hello, ${this.name}`); }
}

const card = new UserCard();
document.getElementById('name')!.textContent = card.name; // interpolation
const btn = document.getElementById('btn') as HTMLButtonElement;
btn.disabled = !card.isActive; // property binding
btn.addEventListener('click', () => card.greet()); // event binding
// Angular automates all of the above and keeps it in sync reactively.

// user-card.component.ts
import { Component, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
```

```

selector: 'app-user-card',
standalone: true,
imports: [FormsModule],
template: `
  <!-- INTERPOLATION -->
  <h2>{{ title }}</h2>
  <p>Items: {{ cartItems.length }}</p>
  <p>{{ user?.displayName ?? 'Guest' }}</p>

  <!-- PROPERTY BINDING -->
  <img [src]="avatarUrl" [alt]="user?.displayName">
  <button [disabled]="isSubmitting">Save</button>
  <td [attr.colspan]="columnSpan">Cell</td>
  <div [class.active]="isActive">Profile</div>
  <div [style.color]="isError ? 'red' : 'green'">Status</div>

  <!-- EVENT BINDING -->
  <button (click)="onSave()">Save</button>
  <button (click)="onButtonClick($event)">Click Me</button>
  <input (keyup.enter)="onEnterPressed()">

  <!-- TWO-WAY BINDING -->
  <input [(ngModel)]="username" placeholder="Enter username">
  <p>Hello, {{ username }}!</p>
`
})
export class UserCardComponent {
  title = 'User Profile';
  cartItems = ['book', 'pen', 'lamp'];
  avatarUrl = 'https://example.com/avatar.jpg';
  isSubmitting = false;
  isActive = true;
  isError = false;
  columnSpan = 2;
  user: { displayName: string } | null = null;
  username = '';

  onSave(): void {
    this.isSubmitting = true;
    setTimeout(() => this.isSubmitting = false, 1500);
  }
  onButtonClick(event: MouseEvent): void {
    console.log('Clicked at:', event.clientX, event.clientY);
  }
  onEnterPressed(): void { console.log('Enter pressed'); }
}

```

Section 4 — Before & After Refactor

```

<!-- BEFORE – three common binding mistakes -->

<!-- disabled="false" still disables – string presence = disabled in HTML -->
<button disabled="{{ isLoading }}">Save</button>

```

```

<!-- fires a broken network request before Angular resolves -->


<!-- $event is a DOM Event object, not a string -->
<input (change)="username = $event">

// AFTER
@Component({
  template: `
    <!-- Property binding passes real boolean to .disabled DOM property -->
    <button [disabled]="isLoading">Save</button>

    <!-- No flicker, no broken request -->
    <img [src]="imageUrl">

    <!-- Two-way binding stays in sync automatically -->
    <input [(ngModel)]="username">

    <!-- When you need the raw value from a native event -->
    <input (change)="onInputChange($event)">
  `
})
export class GoodFormComponent {
  isLoading = true;
  imageUrl = 'https://example.com/photo.jpg';
  username = '';

  onInputChange(event: Event): void {
    this.username = (event.target as HTMLInputElement).value;
  }
}

```

Why it matters: `disabled="{{ false }}"` renders as `disabled="false"` in the DOM — but in HTML the presence of the `disabled` attribute means disabled regardless of its value. Property binding `[disabled]="false"` removes the attribute entirely when false.

Section 5 — Common Mistakes

Mistake 1 — Using interpolation for boolean HTML properties

— `disabled="false"` still disables the button because HTML reads attribute presence, not value.
Fix: `[disabled]="isSubmitting"`.

Mistake 2 — Confusing `$event` type across different event bindings

Native DOM events (click, input, keyup): `$event` is a DOM Event — use `(event.target as HTMLInputElement).value`. `ngModelChange`: `$event` is the raw value. Custom `output()` emitters: `$event` is the emitted type. TypeScript may not catch this if typed as any — silent runtime errors.

Mistake 3 — Calling functions with side effects in interpolation

`{{ getUserFullName() }}` runs on every change detection cycle — potentially hundreds of times per second during user interaction. Fix: use a component property, `computed()` signal, or a pure pipe.

Section 6 — Do's and Don'ts

- DO: `[disabled]="booleanExpression"` for boolean DOM properties
- DON'T: `disabled="{{ booleanExpression }}"` — attribute strings don't map to boolean properties
- DO: `[src]`, `[href]` for dynamic URLs
- DON'T: `src="{{ url }}"` — causes broken request before Angular resolves
- DO: `[(ngModel)]` for simple template-driven inputs
- DON'T: `[(ngModel)]` in reactive forms — conflicts with `FormControl`
- DO: Type `$event` explicitly in event handler methods
- DON'T: Leave event handler params as any — you lose type safety
- DO: `[attr.aria-label]`, `[attr.colspan]` for real HTML attributes with no DOM property
- DON'T: `[attr.disabled]` when you mean `[disabled]` — they behave very differently
- DO: `(keyup.enter)` shorthand for keyboard shortcuts
- DON'T: Write `(keyup)="if($event.key==='Enter') handler()"` in the template

Section 7 — Interview Prep

Question 1

Question: "What is the difference between property binding and attribute binding in Angular? When would you use `[attr.x]` instead of `[x]`?"

Strong answer: HTML attributes set the initial DOM state; DOM properties reflect the current live state. Property binding `[x]` targets the JavaScript property on the DOM element. Attribute binding `[attr.x]` sets the HTML attribute — needed when no matching DOM property exists (e.g. `colspan`, `aria-label`, SVG attributes). `[disabled]` sets the boolean property — Angular removes the attribute when false. `[attr.disabled]` sets the attribute string — "false" still disables. Example: because `colspan` is an attribute, not a DOM property.

Weak answer misses: The distinction between HTML attribute as initialiser vs DOM property as live state.

Level: Mid

Question 2

Question: "How does Angular's two-way binding actually work? Can you implement the banana-in-a-box pattern on a custom component?"

Strong answer: `[(ngModel)]="x"` desugars to `[ngModel]="x" (ngModelChange)="x = $event"`. The pattern works on any component via the `input() x` and `output() xChange` convention — not just native inputs. The parent writes to the input, the child emits on the output, Angular updates the parent property.

Weak answer misses: That `[(x)]` works on any component — candidates think it only works with `ngModel` on native inputs.

Level: Mid-Senior

Section 8 — Quick Reference Card

3-line definition

Angular templates connect a TypeScript class to the DOM using four binding mechanisms. `{{ }}` displays text. `[prop]` sets DOM properties. `(event)` listens for user actions. `[( )]` does both.

Syntax at a glance

Syntax	Direction	Example
-----	-----	-----
<code>{{ x }}</code>	-> DOM	<code>{{ title }}</code>
<code>[property]="x"</code>	-> DOM	<code>[disabled]="loading"</code>
<code>[attr.x]="y"</code>	-> DOM	<code>[attr.colspan]="n"</code>
<code>[class.x]="bool"</code>	-> DOM	<code>[class.active]="isOn"</code>
<code>[style.x]="val"</code>	-> DOM	<code>[style.color]='red'"</code>
<code>(event)="fn()"</code>	DOM ->	<code>(click)="save()"</code>
<code>[(ngModel)]="x"</code>	both	<code>[(ngModel)]="email"</code>

The one rule that matters most

`[disabled]="false"` removes the attribute — the button is enabled. `disabled="false"` keeps the attribute — the button is still disabled. Always use property binding for booleans.

TOPIC 29 — Built-in Control Flow

Category: Angular Core | Difficulty: Beginner-Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

- Difficulty: Beginner-Intermediate — the syntax is straightforward, but understanding when Angular re-renders, how track works, and why the block syntax replaced structural directives requires understanding change detection and DOM reconciliation
- Angular Relevance: Critical — every Angular template that shows dynamic content uses control flow; @if and @for appear in virtually every component you will ever write

Section 1 — Explanation

The Stage Manager Analogy

A spotlight — @if. Either illuminates a scene or it does not. When the condition is false, the actors leave the stage entirely — not just go dark. When true again, actors walk back on fresh. @if does not hide elements, it removes and recreates them.

A photocopier — @for. Hand it one template and a list of 50 names, it stamps out 50 copies. Change the list and it figures out the minimum copies to add, remove, or move. The track instruction tells it how to recognise which copy is which.

A costume rack — @switch. Holds several outfits. Based on the character's role, exactly one comes off the rack. Only one @case is ever active.

Why the change from structural directives

- Old *ngIf/*ngFor required importing CommonModule — new syntax is built into the compiler, no imports needed
- New syntax supports TypeScript type narrowing inside @if blocks
- @for now requires track, making the performance contract explicit
- Block syntax is easier to read for developers who know JavaScript

Old vs New

Old	New	Key Difference
*ngIf="cond"	@if (cond) { }	Type-narrowing, no t
*ngIf="cond; else tmpl"	@if (cond) { } @else { }	Inline else, no ng-t
*ngFor="let x of list; trackBy: fn"	@for (x of list; track x.id) { }	track is mandatory
[ngSwitch] + *ngSwitchCase	@switch (val) { @case (x) { } }	Single coherent bloc

@if — Conditional Rendering

Angular removes DOM nodes when false, creates them fresh when true. Component lifecycle hooks fire — ngOnInit fires every time the component enters the DOM, ngOnDestroy every time it leaves. Type narrowing: inside @if (user) { }, TypeScript knows user is not null — no user!.name needed.

@for — List Rendering

track is mandatory. Angular uses the tracked value to perform minimum DOM operations when the list changes — add, remove, or move nodes rather than destroying and recreating the entire list.

Built-in loop variables

Variable	Type	Meaning
-----	-----	-----
\$index	number	Current index (0-based)
\$first	boolean	True for first item
\$last	boolean	True for last item
\$even	boolean	True for even-index items
\$odd	boolean	True for odd-index items
\$count	number	Total items in list

@switch — Multi-Branch Rendering

Uses strict equality (===). Only one @case renders at a time. No fallthrough — no break needed. @default is the catch-all.

Section 2 — Modern Angular Pattern (v17+)

All three blocks are the v17+ pattern — there is no newer replacement. Key rules:

- No CommonModule, NgIf, or NgFor in imports — control flow is compiler-native
- Always pair @for with track item.id (stable unique property), never track \$index for dynamic lists
- Use @empty inside @for instead of a wrapping @if (items.length > 0)
- Use computed() signals as the data source for @for — memoised, returns stable reference between recalculations
- @if type-narrows TypeScript — no ! assertions needed inside the block

```
// computed() as @for source – correct pattern
visibleItems = computed(() => this.allItems().filter(i => i.visible));

// Template:
// @for (item of visibleItems(); track item.id) { ... }
// NOT: @for (item of getVisibleItems(); track $index) – new array every cycle
```

Section 3 — Code Example

```
// Plain JS – what @for automates
const items = ['Apple', 'Banana', 'Cherry'];
const container = document.getElementById('list');
items.forEach(item => {
  const li = document.createElement('li');
  li.textContent = item;
  container.appendChild(li);
});
// Problem: if items changes, how do you update the DOM efficiently?
// Angular's @for + track handles this diffing automatically.

// dashboard.component.ts – all three blocks
import { Component, signal } from '@angular/core';
```

```

@Component({
  selector: 'app-dashboard',
  standalone: true,
  // No imports needed for control flow
  template: `
    @if (isLoading()) {
      <div class="spinner">Loading...</div>
    } @else if (error()) {
      <div class="error-banner">
        Error: {{ error() }}
        <button (click)="retry()">Retry</button>
      </div>
    } @else {
      <h2>Users ({{ users().length }})</h2>
      <ul>
        @for (user of users(); track user.id) {
          <li [class.inactive]="!user.isActive"
            [class.first-item]="$first">
            <span>{{ $index + 1 }} / {{ $count }}</span>
            <strong>{{ user.name }}</strong>
            @switch (user.role) {
              @case ('admin') { <span class="badge">Admin</span> }
              @case ('editor') { <span class="badge">Editor</span> }
              @default { <span class="badge">Viewer</span> }
            }
          </li>
        } @empty {
          <li class="no-results">No users found.</li>
        }
      </ul>
    }
  `
})
export class DashboardComponent {
  isLoading = signal(true);
  error = signal<string | null>(null);
  users = signal<{id:number,name:string,role:string,isActive:boolean}[]>([]);
  retry(): void { this.isLoading.set(true); this.error.set(null); }
}

// track in depth – why it matters

// WRONG: track $index – items identified by position
// Insert at start: Angular thinks ALL items changed
@for (item of items; track $index) {
  <app-item [data]="item"></app-item>
}

// CORRECT: track item.id – each item has stable identity
// Insert at start: 1 new DOM node created, rest moved
@for (item of items; track item.id) {
  <app-item [data]="item"></app-item>
}

// Primitives: track the value itself

```

```

@for (name of names; track name) {
  <li>{{ name }}</li>
}

```

Section 4 — Before & After Refactor

```

// BEFORE – old structural directive syntax
import { CommonModule } from '@angular/common';

@Component({
  standalone: true,
  imports: [CommonModule], // required for *ngIf/*ngFor/*ngSwitch
  template: `
    <div *ngIf="isLoading; else content">Loading...</div>
    <ng-template #content>
      <div *ngFor="let user of users; trackBy: trackById; let i = index">
        {{ i }}: {{ user.name }}
      </div>
    </ng-template>
    <div [ngSwitch]="user.role">
      <span *ngSwitchCase="'admin'">Admin</span>
      <span *ngSwitchDefault>Viewer</span>
    </div>
  `
})
export class OldComponent {
  trackById(index: number, user: User): number { return user.id; }
}

// AFTER – Angular 17+ built-in control flow
@Component({
  standalone: true,
  imports: [], // nothing needed for control flow
  template: `
    @if (isLoading) {
      <div>Loading...</div>
    } @else {
      @for (user of users; track user.id) {
        {{ $index }}: {{ user.name }}
      } @empty {
        No users.
      }
    }
    @switch (user.role) {
      @case ('admin') { <span>Admin</span> }
      @case ('editor') { <span>Editor</span> }
      @default { <span>Viewer</span> }
    }
  `
})
export class NewComponent {
  isLoading = true;
  users: User[] = [];
  // No trackBy method – track expression is inline

```

}

Why it matters: Fewer imports, better TypeScript integration, mandatory track prevents the most common `@for` performance mistake, `@else` branches are co-located, `@empty` eliminates the wrapping `@if` pattern.

Section 5 — Common Mistakes

Mistake 1 — Using track \$index as the default

For lists that change order or have items inserted mid-list, Angular treats every item at a changed position as a different item — every app-card is destroyed and recreated, firing `ngOnInit` again, losing internal state. Fix: always use `track item.id` unless the list is static and append-only.

Mistake 2 — Expecting @if to hide elements like `display: none`

`@if (false)` removes the DOM node and destroys the component. `ngOnInit` fires on every false to true transition. If `ngOnInit` makes an HTTP call, you get a new API call every time the panel appears. Fix: use `[hidden]="!showPanel"` when you need to preserve component state across toggles.

Mistake 3 — Forgetting that @if removal triggers child `ngOnDestroy`

A component inside `@if` with an unmanaged subscription leaks memory on every toggle. Each time the component mounts, a new subscription is added — memory grows linearly, duplicate side effects fire. Fix: always use `takeUntilDestroyed()` for components inside conditional blocks.

Section 6 — Do's and Don'ts

- DO: track `item.id` (stable unique identity) in `@for`
- DON'T: track `$index` as default for dynamic lists
- DO: `@empty` inside `@for` for empty-state handling
- DON'T: Wrap `@for` in `@if (items.length > 0)` just to handle empty
- DO: Use `@if` type narrowing — no `!` assertions inside block
- DON'T: Assert `obj!.property` inside an `@if (obj)` block — it's redundant
- DO: `[hidden]="condition"` to preserve component state across toggles
- DON'T: Use `@if` when you need hide-but-preserve — they are not the same
- DO: `@switch` for role/status/type enum branching
- DON'T: `@switch` for range comparisons — it uses `===`, not ranges; use `@if/@else if`

Section 7 — Interview Prep

Question 1

Question: "What does the track expression do in Angular's `@for` block, and what happens if you always use `track $index`?"

Strong answer: track gives Angular a way to identify each list item by a stable key across re-renders. Angular uses it for DOM reconciliation — moving/updating existing nodes rather than

destroying and recreating. track \$index identifies items by position — inserting at the start causes all subsequent items to be treated as changed. Child component state is lost, ngOnInit re-fires, animations reset, performance degrades. track \$index is only acceptable for static, append-only lists that never reorder.

Weak answer misses: That track \$index is not always wrong — the real issue is dynamic reordering and insertion.

Level: Mid

Question 2

Question: "What is the difference between @if and [hidden] for conditionally showing content?"

Strong answer: @if (false) removes the DOM node entirely — child component instances destroyed, ngOnDestroy fires, zero memory/CD cost when hidden. [hidden] applies display: none — component instance stays alive, subscriptions active, change detection still runs every cycle. @if is more efficient when content is rarely shown or has expensive ngOnInit side effects. [hidden] preserves component state across toggles. Default choice is @if; switch to [hidden] only for specific state-preservation needs.

Weak answer misses: That [hidden] is not a performance shortcut — it keeps everything alive.

Level: Mid

Question 3

Question: "How does Angular 17's built-in control flow differ from the old structural directives?"

Strong answer: New syntax is compiled directly by the Angular template compiler — no CommonModule or NgIf/NgFor imports needed. @if provides TypeScript type narrowing — old *ngIf did not. @for makes track mandatory — old trackBy was optional and often forgotten. @empty is a clean companion to @for — old approach required a separate *ngIf and ng-template. Old structural directives still work but are the legacy path.

Weak answer misses: The TypeScript type narrowing improvement.

Level: Mid-Senior

Section 8 — Quick Reference Card

3-line definition

@if, @for, and @switch are compiler-native template blocks that control whether DOM nodes exist, how many times they repeat, and which branch renders — no imports required.

Block	Syntax	DOM Behaviour	Key Option
-----	-----	-----	-----
@if	@if (expr) { } @else { }	Creates/destroys	@else if
@for	@for (x of list; track x.id) { }	Stamps per item	@empty
@switch	@switch (val) { @case (x) { } }	One branch	@default

@if vs [hidden]

	@if (cond)	[hidden]="!cond"
-----	-----	-----

DOM hidden	Removed	display: none
Component	Destroyed	Alive
ngOnInit	Fires each time	Fires once ever
Memory/CD	Zero	Full cost
Default	Yes	State preservation only

The one rule that matters most

Always track item.id in @for. track \$index silently destroys and recreates every component when the list reorders — losing state and killing performance.

TOPIC 30 — Built-in Directives

Category: Angular Core | Difficulty: Beginner-Intermediate | Relevance: High

Section 0 — Difficulty & Priority Rating

- Difficulty: Beginner-Intermediate — `ngClass` and `ngStyle` are straightforward but have multiple syntax forms that confuse developers; `ngTemplateOutlet` requires understanding Angular's template reference system
- Angular Relevance: High — `ngClass` and `ngStyle` appear in virtually every real Angular app; `ngTemplateOutlet` is essential for reusable component patterns and dynamic UI

Section 1 — Explanation

The Wardrobe Analogy

`ngClass` is a smart wardrobe assistant. You hand it a list of outfit rules — "wear the red jacket if it's cold, wear the formal shoes if there's a meeting." It looks at all conditions at once and puts on every item whose condition is true — multiple items simultaneously, each independently gated.

`ngStyle` is the same assistant but for fine-grained tailoring — "set the jacket length to exactly 72cm, set the collar width based on today's temperature." You're not toggling items on/off, you're setting precise values dynamically.

`ngTemplateOutlet` is a director calling an actor onto a stage from backstage. The actor (an `ng-template`) is defined somewhere in your template but is not rendered yet. `ngTemplateOutlet` says "bring that actor on now" and optionally hands them a script (a context object with data). The same actor can be called onto multiple stages simultaneously with different scripts each time.

Key distinction

- Structural directives (`@if`, `@for`) — change the DOM structure (add/remove nodes)
- Attribute directives (`ngClass`, `ngStyle`) — change the DOM appearance (modify existing nodes)
- `ngTemplateOutlet` — structural directive that stamps a defined template into the DOM

`ngClass` — input forms

Input Form	Syntax	Use When
String	<code>[ngClass]='class-a class-b'</code>	Adding static classes (prefer <code>class=</code>)
Array	<code>[ngClass]='[class-a, class-b]'</code>	List of always-applied class names
Object	<code>[ngClass]='{ 'a': condA, 'b': condB }'</code>	Most common — conditional classes

Object form is most useful: keys are class names, values are boolean expressions. Every key whose value is truthy gets applied; falsy keys are removed.

`ngStyle`

Accepts an object where keys are CSS property names and values are the style values. The `.px`, `.em`, `.%` unit suffixes are a convenience: `'font-size.px': 14` is equivalent to `'font-size': '14px'`.

`ngTemplateOutlet`

Renders a named ng-template reference at a specific location, optionally passing data via a context object. \$implicit is the default context variable — accessed via let-item (no equals sign). Named context properties are accessed via let-label="label".

Section 2 — Modern Angular Pattern (v17+)

NgClass, NgStyle, and NgTemplateOutlet must be imported from @angular/common — unlike @if/@for, they are not compiler-native.

```
import { NgClass, NgStyle, NgTemplateOutlet } from '@angular/common';

@Component({
  standalone: true,
  imports: [NgClass, NgStyle, NgTemplateOutlet],
  ...
})
```

Modern preference — class bindings

Conditions	Use	Example
1 class	[class.x]	[class.active]="isOn"
2-3 classes	Multiple [class]	[class.a]="x" [class.b]="y"
4+ classes	[ngClass] object	[ngClass]="{ a:x, b:y }"
Computed list	[ngClass] array	[ngClass]="getClasses()"

Modern preference — compute ngClass as computed() signal

```
// Class object as computed signal – recalculates only when deps change
cardClasses = computed(() => ({
  'active': this.user()?.isActive ?? false,
  'premium': this.tier() === 'premium',
}));
// Template: [ngClass]="cardClasses()"
// NOT: [ngClass]="{ 'active': userService.getUser()?.isActive }"
// – service calls in template run every CD cycle
```

Section 3 — Code Example

```
// task-board.component.ts
import { Component, signal } from '@angular/core';
import { NgClass, NgStyle } from '@angular/common';

@Component({
  selector: 'app-task-board',
  standalone: true,
  imports: [NgClass, NgStyle],
  template: `
    @for (task of tasks(); track task.id) {
      <div
        [ngClass]="{
          'task-card':      true,
          'task-selected':  task.isSelected,
          'task-blocked':   task.status === 'blocked',
          'task-done':      task.status === 'done',
        }"
      >
```

```

        'priority-high': task.priority === 'high',
        'priority-critical': task.priority === 'critical'
    }"
    (click)="selectTask(task.id)"
  >
    <h3>{{ task.title }}</h3>
    <div
      class="progress-bar"
      [ngStyle]="{
        'width.%': task.progress,
        'background-color': getProgressColour(task.progress),
        'transition': 'width 0.3s ease'
      }"
    ></div>
    <span [ngClass]="getPriorityClasses(task.priority)">
      {{ task.priority }}
    </span>
  </div>
}
}

})
export class TaskBoardComponent {
  tasks = signal([
    { id: 1, title: 'Design', priority: 'high', status: 'in-progress', isSelected: false, pro
    { id: 2, title: 'Tests', priority: 'medium', status: 'todo', isSelected: false, pro
    { id: 3, title: 'Deploy', priority: 'critical', status: 'blocked', isSelected: true, pro
  ]);
  selectTask(id: number) {
    this.tasks.update(list => list.map(t => ({ ...t, isSelected: t.id === id })));
  }
  getPriorityClasses(p: string): string[] {
    const base = ['badge', `badge-${p}`];
    if (p === 'critical') base.push('badge-pulse');
    return base;
  }
  getProgressColour(n: number): string {
    return n < 30 ? '#ef4444' : n < 70 ? '#f59e0b' : '#22c55e';
  }
}
}

```

ngTemplateOutlet — DataListComponent with input() signals

```

// Reusable list – modern input() signals (not @Input() decorators)
import { Component, input, TemplateRef } from '@angular/core';
import { NgTemplateOutlet } from '@angular/common';

@Component({
  selector: 'app-data-list',
  standalone: true,
  imports: [NgTemplateOutlet],
  template: `
    <ul>
      @for (item of items(); track $index) {
        <li>
          @if (itemTemplate()) {
            <ng-container

```

```

        *ngTemplateOutlet="itemTemplate(!); context: { $implicit: item, index: $index }"
    ></ng-container>
  } @else {
    <span>{{ item }}</span>
  }
</li>
}
</ul>
})
export class DataListComponent<T> {
  items      = input<T[]>([]);
  itemTemplate = input<TemplateRef<{ $implicit: T; index: number }>>();
}

// Parent usage:
// <ng-template #userRow let-user let-i="index">
//   <div>{{ i + 1 }}. {{ user.name }}</div>
// </ng-template>
// <app-data-list [items]="users" [itemTemplate]="userRow"></app-data-list>

```

Section 4 — Before & After Refactor

```

// BEFORE – verbose conditional class logic
<div class="{{ 'card' + (isActive ? ' active' : '') + (isSelected ? ' selected' : '') }}">
  {{ title }}
</div>

// Multiple [class.x] – unmanageable beyond 3-4 classes
<div class="card"
  [class.active]="isActive" [class.selected]="isSelected"
  [class.error]="hasError"  [class.featured]="isFeatured"
  [class.disabled]="isDisabled" [class.loading]="isLoading">
</div>

// Inline style string – unreadable, no type checking
<div [style]="`color: ${textColor};font-size: ${fontSize} px`"></div>

// AFTER – ngClass and ngStyle
import { NgClass, NgStyle } from '@angular/common';

@Component({
  standalone: true,
  imports: [NgClass, NgStyle],
  template: `
    <div
      [ngClass]="{
        'card':      true,
        'active':    isActive,
        'selected':  isSelected,
        'error':     hasError,
        'featured':  isFeatured,
        'disabled':  isDisabled,
        'loading':   isLoading
      }"
    >

```

```

    }"
  >{{ title }}</div>

  <div
    [ngStyle]="{
      'color':      textColor,
      'font-size.px': fontSize,
      'opacity':    isVisible ? 1 : 0
    }"
  ></div>
,
})

```

Why it matters: The object form co-locates all class logic in one binding. Angular diffs the object and only touches the specific classes that changed — it does not rebuild the entire class string on every cycle.

Section 5 — Common Mistakes

Mistake 1 — Using `ngStyle` for static or rarely-changing styles

`[ngStyle]='{ 'background-color': '#3b82f6', 'border-radius': '8px' }'` — these values never change at runtime. `ngStyle` runs on every CD cycle; static styles belong in the component's CSS file where the browser evaluates them once. Reserve `ngStyle` for values computed at runtime from component state.

Mistake 2 — Forgetting to import `NgClass`/`NgStyle` in standalone components

The binding silently fails — no error, no classes applied. Looks identical to a CSS bug. Mental checklist: `@if/@for/@switch` = no import needed. `NgClass`/`NgStyle`/`NgTemplateOutlet` = must import from `@angular/common`.

Mistake 3 — Misunderstanding `$implicit` vs named context in `ngTemplateOutlet`

```

// Passing as named property
*ngTemplateOutlet="tpl; context: { user: currentUser }"
// Trying to access as $implicit — gets undefined
<ng-template #tpl let-user>{{ user?.name }}</ng-template>

// Rule: let-x (no =) captures $implicit
//       let-x="key" captures a named key

```

Fails silently — Angular template renders blank string with no error.

Section 6 — Do's and Don'ts

- DO: `[ngClass]` object form for multiple conditional classes
- DON'T: Build class strings with concatenation and ternaries
- DO: `[class.x]="bool"` for a single class toggle
- DON'T: Reach for `[ngClass]` for a single class
- DO: Import `NgClass`, `NgStyle`, `NgTemplateOutlet` explicitly in standalone components
- DON'T: Assume they work without imports like `@if/@for`

- DO: ngStyle only for runtime-computed values
- DON'T: ngStyle for static styles — put those in CSS
- DO: ng-container as host for *ngTemplateOutlet — no DOM footprint
- DON'T: Use a real element like div as host — adds unnecessary wrapper
- DO: Define ng-template once outside loops
- DON'T: Define ng-template inside @for — creates new TemplateRef per iteration

Section 7 — Interview Prep

Question 1

Question: "What are the different ways to apply classes in Angular? When would you use [class.x] versus [ngClass]?"

Strong answer: Four approaches — class="static", [class.x]="bool", [ngClass]="object/array/string", [class]="expression". [class.x] is ideal for a single conditional class. [ngClass] with an object is ideal for three or more independently conditioned classes. Both can coexist: class="base" [ngClass]="{ active: isActive }". NgClass must be imported in standalone components.

Weak answer misses: That [class.x] and [ngClass] can be combined, and that class="static" is still valid alongside dynamic bindings.

Level: Junior-Mid

Question 2

Question: "How does ngTemplateOutlet work? What is \$implicit?"

Strong answer: ngTemplateOutlet renders a ng-template reference at a specific DOM location — templates are defined but not rendered until explicitly stamped. The context object passes data into the template at render time. \$implicit is the default context key — accessed via let-varName (no equals sign). Named properties require let-varName="keyName". The same template can be stamped multiple times with different contexts. TemplateRef can be passed as input() enabling truly generic, reusable components.

Weak answer misses: The \$implicit vs named context distinction, and the TemplateRef as input() pattern.

Level: Mid-Senior

Question 3

Question: "What is ng-container and why use it as the host for *ngTemplateOutlet?"

Strong answer: ng-container is a grouping element Angular removes from the DOM after compilation — it leaves no DOM trace. Using a div as the host adds an extra wrapper that can break CSS layouts (flexbox, grid, table structure). Also used with structural directives when you need to apply the directive without adding a wrapper element.

Weak answer misses: The CSS layout breakage consequence.

Level: Mid

Section 8 — Quick Reference Card

3-line definition

`ngClass` and `ngStyle` are attribute directives that modify an existing element's classes and styles. `ngTemplateOutlet` stamps a defined `ng-template` into the DOM with optional context data. All three must be imported from `@angular/common`.

Directive	Import From	Input Type	Common Use
-----	-----	-----	-----
<code>NgClass</code>	<code>@angular/common</code>	<code>string string[] object</code>	Multiple conditional classes
<code>NgStyle</code>	<code>@angular/common</code>	<code>{ [prop: string]: any }</code>	Dynamic computed styles
<code>NgTemplateOutlet</code>	<code>@angular/common</code>	<code>TemplateRef null</code>	Template reuse

\$implicit vs named context

```
<!-- $implicit – access with let-x (no equals) -->
*ngTemplateOutlet="tpl; context: { $implicit: item }"
<ng-template #tpl let-item>{{ item.name }}</ng-template>

<!-- Named – access with let-x="key" -->
*ngTemplateOutlet="tpl; context: { label: 'Hello' }"
<ng-template #tpl let-label="label">{{ label }}</ng-template>
```

The one rule that matters most

`NgClass`, `NgStyle`, and `NgTemplateOutlet` will silently do nothing if missing from imports: `[]` — unlike `@if/@for`, they are not compiler-native.

TOPIC 31 — Content Projection

Category: Angular Core | Difficulty: Intermediate | Relevance: High

Section 0 — Difficulty & Priority Rating

- Difficulty: Intermediate — the basic single-slot pattern is simple, but multi-slot projection with select and understanding which context projected content renders in trips up even experienced developers
- Angular Relevance: High — content projection is the mechanism behind every reusable container component — cards, modals, tabs, accordions; without it you cannot build a proper component library

Section 1 — Explanation

The Picture Frame Analogy

A picture frame manufacturer builds beautiful frames — ornate borders, glass, backing, hanging hardware — but has no idea what picture will go inside. The frame has a slot where the picture goes, and whoever buys the frame provides their own picture.

ng-content is that slot. Your component is the frame — it defines structure, styling, and behaviour. The parent provides the picture — the content that goes inside. The frame does not care what the picture is. The picture does not care what frame it is in. They combine cleanly because the slot is the contract between them.

A multi-slot frame is like a shadow box with labelled compartments — "photo goes here, caption goes here, date stamp goes here." The select attribute on ng-content is the label on each compartment.

Three patterns of increasing power

- Single-slot projection — one ng-content — all parent content lands in the one slot
- Multi-slot projection — ng-content select="..." — different parent content routed to different named slots using CSS selectors
- Conditional projection — ng-content inside @if — controls display but NOT instantiation (critical caveat)

The select attribute accepts any CSS selector

Selector Type	Example	Matches
Element	select="header"	<header> elements
Attribute	select="[slot=footer]"	Elements with slot="footer"
CSS class	select=".card-title"	Elements with class card-title
Component	select="app-icon"	<app-icon> component elements

The rendering context rule — the most important concept

Projected content is compiled in the parent's context and rendered in the child's slot.

- Variables, methods, signals in {{ }} inside projected content refer to the parent component's class

- CSS styles from the child component's stylesheet do not apply to projected content (view encapsulation)
- The parent's change detection governs the projected content's bindings

Section 2 — Modern Angular Pattern (v17+)

Content projection syntax is unchanged in v17+. What changes is how the child queries its projected content — `contentChild()` and `contentChildren()` signal functions replace `@ContentChild/@ContentChildren` decorators (see Topic 32).

- `ng-content` with no select — default catch-all slot
- `ng-content select="[attr-name]"` — attribute selectors are most reliable; avoid class selectors which can collide
- Use `ng-container` as the wrapper around `ng-content` when you need structural grouping with no DOM footprint
- When the child needs to pass data into the projected area, or when lazy instantiation is required — switch from `ng-content` to `TemplateRef` + `ngTemplateOutlet` (Topic 30)

```
// Modern modal – attribute-selector multi-slot projection
// Uses input() and output() – no @Input()/@Output() decorators
import { Component, input, output } from '@angular/core';

@Component({
  selector: 'app-modal',
  standalone: true,
  template: `
    <div class="modal-overlay" (click)="handleOverlayClick()">
      <div class="modal-shell" (click)="$event.stopPropagation()">
        <div class="modal-header">
          <ng-content select="[modal-title]"></ng-content>
          <button (click)="closed.emit()">X</button>
        </div>
        <div class="modal-body">
          <ng-content select="[modal-body]"></ng-content>
        </div>
        <div class="modal-footer">
          <ng-content select="[modal-footer]"></ng-content>
        </div>
      </div>
    </div>
  `
})
export class ModalComponent {
  size = input<'sm' | 'md' | 'lg'>('md');
  closeOnOverlay = input(true);
  closed = output<void>();

  handleOverlayClick(): void {
    if (this.closeOnOverlay()) this.closed.emit();
  }
}
```

Section 3 — Code Example

```
// Single-slot projection
@Component({
  selector: 'app-card',
  standalone: true,
  template: `
    <div class="card">
      <div class="card-shell">
        <!-- Whatever the parent puts between <app-card> tags lands here.
             CardComponent has no idea what it is – nor should it. -->
        <ng-content></ng-content>
      </div>
    </div>
  `
})
export class CardComponent {}

// Parent usage – bindings resolve against PARENT's class
@Component({
  standalone: true,
  imports: [CardComponent],
  template: `
    <app-card>
      <!-- userName and viewMessages() come from PageComponent, not CardComponent -->
      <h3>Welcome back, {{ userName }}!</h3>
      <p>You have {{ messageCount }} unread messages.</p>
      <button (click)="viewMessages()">View Messages</button>
    </app-card>
  `
})
export class PageComponent {
  userName = 'Alice';
  messageCount = 4;
  viewMessages(): void { console.log('Viewing messages'); }
}

// Multi-slot projection
@Component({
  selector: 'app-panel',
  standalone: true,
  template: `
    <div class="panel">
      <div class="panel-header">
        <ng-content select="[panel-header]"></ng-content>
      </div>
      <div class="panel-body">
        <ng-content select=".panel-body"></ng-content>
      </div>
      <!-- Default – catches everything NOT matched above -->
      <div class="panel-default">
        <ng-content></ng-content>
      </div>
    </div>
  `
})
```

```

})
export class PanelComponent {}

// Parent usage
// <app-panel>
//   <h2 panel-header>Dashboard Overview</h2> <- attribute -> SLOT 1
//   <div class="panel-body"><p>Content</p></div> <- class -> SLOT 2
//   <p>No selector – falls to default slot.</p> <- DEFAULT
// </app-panel>

// Conditional slot – IMPORTANT: @if controls display, NOT instantiation
@Component({
  selector: 'app-collapsible',
  standalone: true,
  template: `
    <button (click)="isOpen = !isOpen">Toggle</button>
    <!-- Even when isOpen is false, projected content IS instantiated.
         ngOnInit on any projected component fires immediately on parent mount.
         Only DISPLAY is conditional – not component lifecycle. -->
    @if (isOpen) {
      <div class="content"><ng-content></ng-content></div>
    }
  `
})
export class CollapsibleComponent { isOpen = false; }

// For true lazy init: use TemplateRef input instead
// @Input() contentTemplate?: TemplateRef<void>;
// @if (isOpen) { <ng-container *ngTemplateOutlet="contentTemplate"> }

```

Section 4 — Before & After Refactor

```

// BEFORE – @Input() strings for rich content
@Component({
  selector: 'app-modal',
  standalone: true,
  template: `
    <div class="modal">
      <h2>{{ title }}</h2>      <!-- plain string only -->
      <p>{{ body }}</p>        <!-- no HTML, no components, no bindings -->
      <div>{{ footer }}</div>  <!-- no real buttons possible -->
    </div>
  `
})
export class ModalComponent {
  @Input() title = '';
  @Input() body = '';
  @Input() footer = '';
}

// Problems:
// 1. Body cannot contain <a>, <strong>, or another component
// 2. Footer cannot have a real <button> with (click) binding
// 3. Every new content need = new @Input() on the component

```

```
// AFTER – content projection with input()/output() (modern Angular)
import { Component, input, output } from '@angular/core';

@Component({
  selector: 'app-modal',
  standalone: true,
  template: `
    <div class="modal-overlay" (click)="onOverlayClick()">
      <div class="modal" (click)="$event.stopPropagation()">
        <div class="modal-header">
          <ng-content select="[modal-title]"></ng-content>
          <button (click)="close()">X</button>
        </div>
        <div class="modal-body">
          <ng-content select="[modal-body]"></ng-content>
        </div>
        <div class="modal-footer">
          <ng-content select="[modal-footer]"></ng-content>
        </div>
      </div>
    </div>
  `
})
export class ModalComponent {
  closed = output<void>();
  close(): void { this.closed.emit(); }
  onOverlayClick(): void { this.close(); }
}

// Parent – rich content with real HTML and Angular bindings
// <app-modal (closed)="showModal = false">
//   <h2 modal-title>Delete <strong>{{ itemName }}</strong>?</h2>
//   <div modal-body>
//     <p>This will permanently remove {{ itemName }}.</p>
//     <a href="/help/delete">Learn more</a>
//   </div>
//   <div modal-footer>
//     <button (click)="cancel()">Cancel</button>
//     <button (click)="confirmDelete()">Delete</button>
//   </div>
// </app-modal>
```

Why it matters: Content projection makes components composable. The modal provides structure, overlay behaviour, and close logic. The parent provides the specific content. The same modal handles every scenario without ever being modified.

Section 5 — Common Mistakes

Mistake 1 — Assuming projected content resolves against the child component

```
// Child component
export class WrapperComponent {
  childMessage = 'Hello from the child';
}
```

```
// Parent template
<app-wrapper>
  <!-- Developer expects childMessage from WrapperComponent -->
  <p>{{ childMessage }}</p> <!-- resolves against PARENT – renders blank -->
</app-wrapper>
```

Projected content compiles in the parent, renders in the child. Fails silently — no error, blank output. If you need child data in projected content, use `ngTemplateOutlet` with a context object.

Mistake 2 — Expecting @if around ng-content to delay component instantiation

Projected content is part of the parent's template — it instantiates when the parent mounts, not when the child's `@if` becomes true. The child's `@if` only controls display. `ngOnInit` on projected components fires immediately regardless. For true lazy init, use `TemplateRef` input — the child stamps it only when ready.

Mistake 3 — Using ng-content inside @for to repeat projected content

`ng-content` moves DOM nodes once — it cannot copy them. Attempting to project content once per list item produces the content only once. For repeated stamping with data context, use `TemplateRef` + `ngTemplateOutlet`.

Section 6 — Do's and Don'ts

- DO: Use `ng-content` for container components that provide structure but not specific content
- DON'T: Use `@Input() title: string` when content needs rich HTML with components or event bindings
- DO: Use `select="[attr-name]"` attribute selectors for multi-slot projection
- DON'T: Use class selectors for projection slots in large apps — classes collide
- DO: Add a default `ng-content` (no select) as a catch-all
- DON'T: Let unmatched projected content disappear silently
- DO: Use `TemplateRef` + `ngTemplateOutlet` when child needs to pass data into projected content
- DON'T: Try to access child component properties from projected content — impossible with basic projection
- DO: Style the slot wrapper element in child CSS — inherited CSS cascades in
- DON'T: Add child-scoped styles targeting projected elements — view encapsulation prevents them

Section 7 — Interview Prep

Question 1

Question: "What is content projection in Angular and how does `ng-content` work? What does `select` do?"

Strong answer: Content projection allows a parent to pass HTML/components into a child's template via `ng-content` slots. The child defines where content appears; the parent defines what it is. `select` accepts a CSS selector to route specific parent content to specific slots. Content not matching any `select` goes to the default `ng-content` (no select). Content not matched by any slot

and with no default slot is silently discarded. Real example: modal with [modal-title], [modal-body], [modal-footer] slots.

Weak answer misses: What happens to unmatched content, and the practical impact of having no default slot.

Level: Junior-Mid

Question 2

Question: "Which component's context do projected bindings resolve against — parent or child? Why does this matter?"

Strong answer: Projected content bindings always resolve against the parent component. Rule: "compiled in the parent, rendered in the child." {{ variable }} inside projected content refers to the parent's property. The child cannot expose its data to projected content. This is by design — the parent defines what's shown, the child defines where. When child-to-content data flow is needed, TemplateRef + ngTemplateOutlet is the correct pattern — it can pass context data from the child into the parent's template.

Weak answer misses: The architectural implication — ng-content creates a passive child (display contract); TemplateRef creates an active child (rendering contract).

Level: Mid-Senior

Question 3

Question: "If you put @if (isOpen) around ng-content in a child, does that prevent the projected component from initialising until isOpen is true?"

Strong answer: No. Projected content is part of the parent's template and instantiates with the parent. The child's @if only controls display — the content is already created before @if evaluates. ngOnInit in a projected component fires immediately on parent mount. This matters for tabs, accordions, and wizard flows. Fix: use TemplateRef input — pass a blueprint, not live content — so the child controls instantiation.

Weak answer misses: That projected content is owned by the parent's template.

Level: Senior

Section 8 — Quick Reference Card

3-line definition

ng-content is a slot placeholder — the parent writes the content, the child provides the frame. Projected content always resolves bindings against the parent. The child cannot pass data into projected content without switching to TemplateRef.

Pattern	Syntax	Use Case
Single slot	<ng-content>	Simple container components
Named slot	<ng-content select="[a]">	Multi-region layouts
Default catch-all	<ng-content> after named	Catch unmatched content
Conditional display	@if (open) { <ng-content> }	Toggle visibility (NOT init)
Repeated with data	TemplateRef + ngTemplateOutlet	Rows, cells, repeated items

ng-content vs TemplateRef

	ng-content	TemplateRef + ngTemplateOutlet
-----	-----	-----
Parent defines	Live HTML	Unrendered blueprint
Child can pass data	No	Yes (via context)
Lazy instantiation	No (always w/parent)	Yes (child controls when)
Multiple stamps	Once only	Unlimited
Best for	Modals, cards	Data grids, lazy tabs

The one rule that matters most

Projected content always compiles in the parent's context — `{{ x }}` in projected content reads the parent's `x`, never the child's. `@if` around `ng-content` hides content but never delays its initialisation.

TOPIC 32 — ViewChild, ViewChildren, ContentChild

Category: Angular Core | Difficulty: Intermediate | Relevance: High

Section 0 — Difficulty & Priority Rating

- Difficulty: Intermediate — the function syntax is simple, but understanding when signal queries resolve, the difference between the four query functions, and using array-returning signals correctly requires solid understanding of Angular's rendering pipeline
- Angular Relevance: High — used in every non-trivial Angular component needing programmatic access to child elements, child components, or projected content

Section 1 — Explanation

The Theatre Director Analogy

`ViewChild()` is the director having a direct line to one specific actor in their own production — an actor they personally cast. A direct, live reference. The director can tell that actor to do anything.

`ViewChildren()` is the same director asking for all actors of a certain type — they hand back a roster as a reactive signal containing an array.

`ContentChild()` is the director reaching into the guest actor area — actors a guest director (the parent component) brought in via content projection.

`ContentChildren()` is the same but for all guest actors of a given type.

The View vs Content distinction is unchanged. What changed is timing safety — query results are available as soon as you call the signal function. You no longer need to wait for a specific lifecycle hook.

Decorator API vs Signal API

Aspect	Decorator (legacy)	Signal (current)
Declaration	@ViewChild('ref') foo!: ElementRef	foo = viewChild<ElementRef>('ref')
Reading value	this.foo (property)	this.foo() (function call)
Multiple results	QueryList<T>	Signal<readonly T[]>
Safe to read	Only after <code>ngAfterViewInit</code>	Anywhere — returns <code>undefined/[]</code>
React to changes	<code>queryList.changes.subscribe(...)</code>	<code>effect(() => { foo(); ... })</code>
Static option	Required for elements outside <code>@if</code>	Not needed — no static mode

Quick Reference

Function	Searches	Returns
<code>ViewChild(ref)</code>	Own template	Signal<T undefined>
<code>ViewChild.required(r)</code>	Own template	Signal<T> (throws if missing)
<code>ViewChildren(ref)</code>	Own template	Signal<readonly T[]>
<code>ContentChild(ref)</code>	Projected content	Signal<T undefined>
<code>ContentChild.required</code>	Projected content	Signal<T> (throws if missing)
<code>ContentChildren(ref)</code>	Projected content	Signal<readonly T[]>

Section 2 — Modern Angular Pattern (v17+)

Signal queries are the v17+ default. The `{ static: true/false }` option is obsolete — signal queries are lazily evaluated and always return the current state. Use `.required()` variants only for elements unconditionally present in the template.

`effect()` replaces two separate lifecycle patterns in one:

- `ngAfterViewInit / ngAfterContentInit` for initial safe read
- `QueryList.changes.subscribe()` for reacting to later updates

```
export class TabGroupComponent {
  tabs = contentChildren(TabComponent); // Signal<readonly TabComponent[]>

  constructor() {
    // Runs once on resolution, re-runs on every change –
    // replaces ngAfterContentInit + QueryList.changes.subscribe() combined
    effect(() => {
      const allTabs = this.tabs();
      if (allTabs.length > 0 && !this.activeTabId()) {
        this.activeTabId.set(allTabs[0].tabId());
      }
    });
  }
}
```

Section 3 — Code Example

```
// Plain JS – what Angular query functions automate
const inputEl = document.querySelector('#myInput');
inputEl.focus();
// Problems:
// 1. Returns null until element exists – Angular returns undefined safely
// 2. Searches entire document – Angular scopes to this component only
// 3. Returns a static snapshot – Angular signal queries are live
// 4. Returns HTMLElement – Angular returns the typed component instance

// viewChild() with template reference and component type
import { Component, viewChild, signal, ElementRef } from '@angular/core';

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <button (click)="decrement()">-</button>
    <span>{{ count() }}</span>
    <button (click)="increment()">+</button>
  `
})
export class CounterComponent {
  count = signal(0);
  increment(): void { this.count.update(n => n + 1); }
  decrement(): void { this.count.update(n => n - 1); }
  reset(): void { this.count.set(0); }
  getValue(): number { return this.count(); }
}
```

```

@Component({
  selector: 'app-game',
  standalone: true,
  imports: [CounterComponent],
  template: `
    <input #scoreInput type="number" placeholder="Set score">
    <app-counter #scoreCounter></app-counter>
    <button (click)="focusInput()">Focus Input</button>
    <button (click)="resetScore()">Reset Score</button>
  `
})
export class GameComponent {
  // Signal<ElementRef<HTMLInputElement> | undefined>
  scoreInputRef = viewChild<ElementRef<HTMLInputElement>>('scoreInput');
  // Signal<CounterComponent | undefined>
  counterComponent = viewChild(CounterComponent);

  // No ngAfterViewInit needed – call anywhere, returns undefined until resolved
  focusInput(): void { this.scoreInputRef()?.nativeElement.focus(); }
  resetScore(): void { this.counterComponent()?.reset(); }
}

// viewChildren() – signal array, replaces QueryList
export class CardGridComponent {
  cards = signal([
    { id: 1, title: 'One' },
    { id: 2, title: 'Two' }
  ]);

  // Signal<readonly CardComponent[]> – auto-syncs as @for adds/removes
  cardComponents = viewChildren(CardComponent);

  constructor() {
    // effect() replaces QueryList.changes.subscribe()
    effect(() => console.log('Card count:', this.cardComponents().length));
  }

  highlightAll(): void { this.cardComponents().forEach(c => c.highlight()); }
  // Plain array indexing – no .first/.last from QueryList
  highlightFirst(): void { this.cardComponents()[0]?.highlight(); }
}

// contentChildren() – projected content queries
export class TabGroupComponent {
  activeTabId = signal('');

  // Queries PROJECTED TabComponent instances
  tabs = contentChildren(TabComponent);

  constructor() {
    effect(() => {
      const allTabs = this.tabs();
      if (allTabs.length > 0 && !this.activeTabId()) {
        this.activeTabId.set(allTabs[0].tabId());
        allTabs[0].activate();
      }
    });
  }
}

```

```

selectTab(tabId: string): void {
  this.tabs().forEach(tab => tab.deactivate());
  const selected = this.tabs().find(t => t.tabId() === tabId);
  if (selected) { selected.activate(); this.activeTabId.set(tabId); }
}
}

```

Section 4 — Before & After Refactor

```

// BEFORE – decorator-based query
import { Component, ViewChild, AfterViewInit, ElementRef } from '@angular/core';

@Component({
  standalone: true,
  template: `<input #emailField type="email"><button (click)="focus()">Focus</button>`
})
export class LoginComponent implements AfterViewInit {
  @ViewChild('emailField')
  emailInput!: ElementRef<HTMLInputElement>; // ! lies – undefined until hook fires

  ngAfterViewInit(): void {
    // MUST be in this hook – reading earlier throws
    this.emailInput.nativeElement.focus();
  }
  focus(): void { this.emailInput.nativeElement.focus(); }
}

// AFTER – signal-based query
import { Component, viewChild, ElementRef } from '@angular/core';

@Component({
  standalone: true,
  template: `<input #emailField type="email"><button (click)="focus()">Focus</button>`
})
export class LoginComponent {
  // No lifecycle hook, no ! assertion, no timing concern
  emailInput = viewChild<ElementRef<HTMLInputElement>>('emailField');

  focus(): void {
    this.emailInput()?.nativeElement.focus(); // optional chaining handles undefined
  }
}

```

Why it matters: Signal queries remove an entire category of timing bugs by construction. The signal's type honestly reflects that the value may be undefined. `viewChild.required()` is available when you want Angular to throw clearly if the element is absent.

Section 5 — Common Mistakes

Mistake 1 — Reading the signal without calling it

```

emailInput = viewChild<ElementRef>('emailField');

// WRONG – this.emailInput is the signal itself, not the value

```

```
this.emailInput.nativeElement.focus(); // TypeError

// CORRECT – call it to get the current value
this.emailInput()?.nativeElement.focus();
```

Mistake 2 — Treating viewChildren() result like a QueryList

```
cardComponents = viewChildren(CardComponent);

// WRONG – QueryList API does not exist on signal result
this.cardComponents.first?.highlight();
this.cardComponents.changes.subscribe(...);

// CORRECT – plain array indexing + effect()
this.cardComponents()[0]?.highlight();
effect(() => console.log(this.cardComponents().length));
```

Mistake 3 — Using viewChild.required() on a conditionally rendered element

```
// WRONG – element is inside @if – may not exist – Angular throws
panelRef = viewChild.required<ElementRef>('panel');
// Template: @if (showPanel) { <div #panel></div> }

// CORRECT – use optional form for conditional elements
panelRef = viewChild<ElementRef>('panel');
this.panelRef()?.nativeElement... // optional chaining handles absent case
```

Reserve `.required()` for elements unconditionally present in the template.

Section 6 — Do's and Don'ts

- DO: Call query signals as functions: `this.myRef()`
- DON'T: Read `this.myRef` as a property — it's the signal, not the value
- DO: Use `effect()` to react to query resolution and changes
- DON'T: Read queries once in the constructor and assume the value is final
- DO: Use `.required()` only for unconditionally present elements
- DON'T: Use `.required()` on elements inside `@if/@for` — it throws
- DO: Use plain array indexing on `viewChildren()` results: `[0]`, `.find()`, `.forEach()`
- DON'T: Use `.first`, `.last`, `.toArray()`, `.changes` — these are QueryList APIs that do not exist on signal arrays
- DO: Prefer `input()/output()` for data flow
- DON'T: Reach for `viewChild()` to read child state — use `output()` instead
- DO: Use `contentChildren()` + `effect()` for interactive containers

Section 7 — Interview Prep

Question 1

Question: "What is the difference between `viewChild()` and `@ViewChild()`? Why did Angular introduce the signal-based version?"

Strong answer: `viewChild()` is a function called at field-initialization time, returning `Signal`. `@ViewChild` is a decorator that assigns a class property. Reading the signal requires calling it: `this.myRef()`. Signal queries do not require a specific lifecycle hook — they return undefined until resolved, then stay live. The `{ static: true }` option is unnecessary with signals. Signal queries integrate naturally with `effect()` and `computed()`, replacing `QueryList.changes`. The View/Content distinction and underlying DOM-querying behaviour are unchanged — only the API surface and safety guarantees differ.

Weak answer misses: That the structural distinction between `viewChild` and `contentChild` is unchanged — only the timing-safety concern is removed.

Level: Mid

Question 2

Question: "How do you react to a `viewChildren()` result changing over time?"

Strong answer: `viewChildren()` returns `Signal`. Wrap the read in `effect()` to automatically re-run whenever the array changes — this replaces `QueryList.changes.subscribe()`. `effect()` requires no manual unsubscription, unlike the `Subscription` returned by `.changes.subscribe()`. The result is a plain array — no `.first/.last/.toArray()` from `QueryList`; use standard array indexing and methods.

Weak answer misses: That `effect()` auto-cleans up on component destroy, unlike a manual subscription.

Level: Mid

Question 3

Question: "What replaced `{ static: true }` in the signal query API?"

Strong answer: `.required()` variants — `viewChild.required()`, `contentChild.required()` — are the closest equivalent. Unlike `{ static: true }`, which silently gave undefined if misused on a conditional element, `.required()` changes the return type to non-optional and throws a clear runtime error if the element is absent. The non-required form works correctly for both conditional and unconditional elements without any configuration. This removes the entire bug class of "used `static: true` on a conditional element and got an intermittent failure."

Weak answer misses: That `.required()` is a type-safety and error-clarity upgrade — the return type changes from `Signal` to `Signal`.

Level: Mid-Senior

Section 8 — Quick Reference Card

3-line definition

Signal queries give you reactive references to elements and components in your template or projected content. All four functions return signals — call them to get the current value. No lifecycle hook is needed; `effect()` handles reacting to changes.

Migration reference

Decorator (legacy)	Signal Function (current)
-----	-----

```

@ViewChild('x') y!: T | y = viewChild<T>('x')
@ViewChild('x', { static: true }) y!: T | y = viewChild.required<T>('x')
@ViewChildren(X) y!: QueryList<X> | y = viewChildren(X)
@ContentChild(X) y?: X | y = contentChild(X)
@ContentChildren(X) y!: QueryList<X> | y = contentChildren(X)
ngAfterViewInit() { this.y... } | Read y() anywhere; no hook needed
queryList.changes.subscribe(...) | effect(() => { y(); ... })

```

Four query functions at a glance

Function	Searches	Quantity	Returns
viewChild()	Own template	First match	Signal<T undefined>
viewChild.required()	Own template	First match	Signal<T> (throws if absent)
viewChildren()	Own template	All matches	Signal<readonly T[]>
contentChild()	Projected content	First match	Signal<T undefined>
contentChild.required	Projected content	First match	Signal<T> (throws if absent)
contentChildren()	Projected content	All matches	Signal<readonly T[]>

The one rule that matters most

Signal queries are read by calling them as functions — `this.myRef()`. Wrap any logic that depends on the resolved value in `effect()` so it re-runs when view creation or content projection actually completes.